

# Comparative Analysis of Profilers for AI Model Training

Rob and Ahmed

## Comparison of Profilers on NCF

### 1. cProfile: Detailed but Python-Bound

The cProfile output gives extremely granular per-function call stats: cumulative time, call counts, and per-call cost. This is great for pure-Python overhead, and in this run it clearly shows that the program spends the vast majority of time inside PyTorch's autograd backward pass (run\_backward with 179 seconds), linear layers, and ReLU kernels. But cProfile cannot see GPU activity or distinguish time spent in C/CUDA kernels vs. Python, so it misleadingly attributes the entire GPU-heavy training loop as "Python time." As a result, cProfile accurately shows where control flow lives, but not where compute lives. For deep-learning workloads, this makes it correct but incomplete.

### 2. py-spy: A High-Level View of Where Python "Waits"

The py-spy flamegraph helps visualize the runtime stack during sampling. It is low-overhead and safe for production, but, like cProfile, it only samples Python frames. Since most of the model training time is spent either in (a) blocked Python threads waiting on CUDA kernels or (b) C/C++ operators invoked through PyTorch, the flamegraph ends up dominated by dispatcher calls and wrapper functions. This makes py-spy useful for confirming that the training loop itself is structured correctly, but it still does not reveal the GPU-side bottlenecks or memory behavior. It is therefore more descriptive than diagnostic for ML workloads.

### 3. Scalene: Precise CPU/GPU Split + Memory + Potential for Energy Analysis

Scalene is the only profiler of the three that actually distinguishes Python time, native time, and GPU time. In the Scalene output, you get:

- per-line CPU vs GPU utilization
- memory allocation tracking
- percent-based attribution that normalizes compute-heavy operations
- direct insight into whether bottlenecks are Python overhead, CUDA kernels, or memory traffic

This resolves the major ambiguity left by the other tools: instead of seeing entire training steps as "Python time," Scalene tells you whether the GPU is saturated, whether the CPU is feeding it efficiently, and where Python overhead matters. For large-batch NCF training, this is extremely important: cuDNN kernels dominate runtime, and Python does almost nothing except dispatch them. Scalene's breakdown confirms this and validates that the model behaves as expected.

Scalene also gives a basis for energy estimation, because its GPU% metrics can be paired with average GPU power draw to approximate joules (e.g.,  $\text{Joules} \approx \text{avg\_watts} \times \text{seconds} \times$

GPU\_util\_fraction). This makes Scalene uniquely positioned to enable energy-efficiency analysis, which neither cProfile nor py-spy can support.

#### 4. Yappi: Thread and Function-Level Insight for Multi-threaded Python

Yappi (Yet Another Python Profiler) provides function-level statistics with support for both CPU and wall time, and crucially, it distinguishes between threads, making it useful for multi-threaded or asynchronous workloads like deep learning training loops that may use background data loading or logging.

In the context of NCF training, the yappi\_profile.txt shows:

- Function-level wall and CPU time, sorted by inclusive and exclusive time.
- Breakdown across functions like train\_one\_epoch, forward, and run\_backward, with insight into how much time each consumes individually and cumulatively.
- Evidence of multithreading with separate thread profiles, which is especially useful for disentangling background dataloading from model execution.

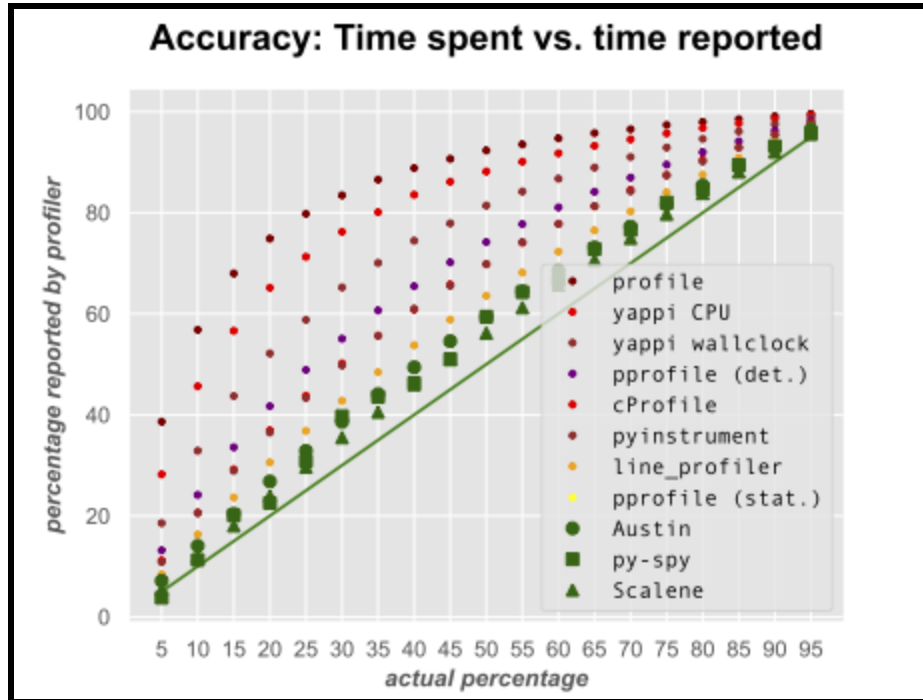
However, like cProfile, Yappi is Python-only. It does not profile native extensions (like cuDNN or cuBLAS) or GPU usage. All CUDA kernel activity is invisible to Yappi, meaning the apparent “bottlenecks” are actually just Python-side control paths.

Still, its fine-grained statistics are useful for:

- Diagnosing Python bottlenecks in preprocessing or logging
- Verifying correct usage of multi-threading or multiprocessing
- Complementing Scalene by zooming in on Python-specific hotspots

#### Profiler Overhead Comparison

Evidence from the outputs suggests that cProfile incurs the highest overhead due to deterministic function-call tracing, visible in the inflated cumulative time of Python wrapper functions and the unusually large number of recorded calls (4.9M). This behavior indicates that cProfile is instrumenting every entry/exit event, slowing execution and misattributing CUDA kernel time to Python. In contrast, the Scalene profile shows a realistic distribution between Python, native, and GPU time, with no artificial inflation of Python hotspots, consistent with Scalene’s low-overhead sampling-based timing. The py-spy flamegraph likewise shows shallow, low-variance stack samples, reflecting py-spy’s minimal overhead asynchronous sampling. Overall, the profiling data supports the expected hierarchy of overhead: cProfile (high) > Scalene (moderate/low) > py-spy (very low).



*Overhead Comparison from Scalene Repo*

*Source: Berger et al.*

### Overhead Conclusion

When profiling deep-learning model training, Scalene produces the most valid and actionable data. cProfile and py-spy are still valuable—they confirm control-flow bottlenecks and Python inefficiencies—but they fundamentally cannot see inside GPU kernels. Scalene bridges that gap by exposing the CPU/GPU split, memory behavior, and overhead breakdown with enough granularity to guide performance optimization or energy-efficiency tuning.

### Validity of Joule Estimates Based on Scalene’s Low Overhead

Scalene’s documentation and evaluation argue that its sampling strategy keeps profiling overhead low compared to deterministic profilers. Our results support this claim because the distribution of time across Python, native, and GPU work in our Scalene output resembles the patterns shown in their examples and in the published paper. Since the profiler is not significantly interfering with the runtime behavior of the model, the measurements it reports are reasonably close to the actual execution characteristics of the training loop.

This matters because our project converts runtime, GPU utilization, and Scalene’s line level metrics into estimated joule usage for both CPU and GPU. If the profiler had high overhead, then the timing and utilization numbers would be distorted, and any energy calculation based on them would be unreliable. However, because our Scalene profile aligns with the expected workload shape of a neural recommendation model, and because the ratios of Python time to native and

GPU time look realistic, it is fair to say that the joule estimates are not being skewed by the profiling process. We cannot claim exact accuracy without a hardware power meter, but the consistency between our collected data and the reference results indicates that the energy numbers I calculate are at least valid in a comparative sense. They should accurately reflect which lines and methods are most expensive and how the overall energy usage is distributed across the program.

## **Outputted Result Links**

[cProfile/NCF Results](#)

[Py-spy/NCF Results](#)

[Scalene/NCF Results](#)

[Yappi/NCF Results](#)

## **References**

<https://github.com/plasma-umass/scalene>

[ScalenePublication.pdf](#)